

TECHNICAL REFERENCE GUIDE

How osquery Works at System Level

Device information extraction, event collection, and a practical Python implementation architecture

Purpose

Explain how osquery turns operating-system state into queryable virtual tables, and provide a realistic design for building equivalent inventory and telemetry capabilities in Python.

Inventory Online / Endpoint Discovery Architecture

7 August 2026

Executive Overview

osquery is a lightweight operating-system instrumentation framework. It exposes device information—processes, users, software, services, hardware, ports, certificates, files and security events—as SQL-style virtual tables. Most tables are generated on demand from native operating-system APIs; event tables buffer transient activity so that short-lived changes are not missed.

Recommended direction

For Inventory Online, retain osquery for low-level, cross-platform collection and build the differentiated control plane in Python: enrolment, scheduling, normalization, asset correlation, secure ingestion, search, reporting and policy management.

Component	Role
osqueryi	Interactive local SQL console for investigation and testing.
osqueryd	Long-running service that schedules queries, stores state and emits results.
Virtual tables	Adapters that translate SQL requests into OS-specific data collection.
Event publishers/subscribers	Capture and buffer transient operating-system activity.
Plugins/extensions	Replace or extend configuration, logging, distributed queries and tables.

1. osquery System-Level Architecture

1.1 The operating system is presented as a database

osquery embeds a SQL query engine, but most tables are not conventional stored tables. A table implementation is a plugin. When a query references a table, the plugin calls the relevant operating-system interface and converts the result into rows and columns.

1	2	3	4	5
SQL query	SQLite planner	Table plugin	Native OS API	Result rows

```
SELECT pid, name, path, on_disk
FROM processes
WHERE on_disk = 0;
```

The query planner can pass useful constraints—such as a PID or pathname—to a table plugin. A well-designed collector uses those constraints to avoid scanning the entire system.

1.2 Interactive shell versus daemon

Capability	osqueryi	osqueryd
Primary use	Ad-hoc inspection	Continuous fleet monitoring
Execution	User enters SQL	Configured schedules and distributed queries
State	Short-lived	Persistent local state
Event capture	Limited by shell lifetime	Continuous while service runs
Output	Console formats	Structured logs or remote logger plugin

1.3 Agent lifecycle on a device

1. Install as a Windows Service, Linux daemon/systemd unit, or macOS LaunchDaemon.
2. Enrol or load local/remote configuration and validate flags, schedules and packs.
3. Initialize table, event, storage, configuration and logging plugins.
4. Execute scheduled queries with performance and result-size controls.
5. Calculate snapshot or differential results and write them to configured loggers.
6. Receive configuration updates or distributed queries from the fleet control plane.

2. How Device Information Is Extracted

There is no single extraction mechanism. Every table chooses an OS-specific source with the required fidelity and privilege level.

Information	Windows	Linux	macOS
Processes	Process APIs, handles, WMI/CIM	/proc and procs	libproc and system APIs
Installed software	Registry, MSI, Store packages	dpkg, RPM and package DBs	App bundles and package receipts
Services	Service Control Manager	systemd / init	launchd
Users/groups	Account and security APIs	NSS, passwd/group	OpenDirectory
Network	IP Helper and network APIs	/proc/net and netlink	System networking APIs
Hardware	WMI/CIM and SetupAPI	sysfs, DMI and procs	IOKit and system APIs
Certificates	Certificate Store	Certificate paths/stores	Keychain APIs
Security events	ETW and Event Log	audit, inotify, eBPF	EndpointSecurity and FSEvents

2.1 Snapshot tables

Snapshot tables describe the state at query time. They are efficient for inventory but can miss a process or connection that begins and ends between executions.

```
SELECT * FROM system_info;
SELECT * FROM users;
SELECT * FROM listening_ports;
SELECT * FROM interface_addresses;
```

2.2 Event-based tables

Event publishers run continuously or register native callbacks. Subscribers select relevant events, transform them into rows and buffer them in osquery's backing storage. Scheduled queries read the buffered rows through tables whose names commonly end in `_events`.

Event source	Publisher	Subscriber/table	Buffered output
OS callback or watcher	Receives raw activity	Filters and normalizes	Rows queried by schedule

```
SELECT pid, path, cmdline, time
FROM process_events
WHERE time > (strftime('%s','now') - 300);
```

Important limitation

A periodic snapshot cannot guarantee detection of short-lived processes, sockets or file changes. Real-time fidelity requires native event mechanisms and appropriate privileges.

2.3 Differential results

For scheduled snapshot queries, the daemon can retain the previous result and emit changes instead of retransmitting the entire dataset. New rows are additions; disappeared rows are removals. This reduces network volume and makes inventory change timelines easier to build.

3. Building Similar Functionality in Python

Python can reproduce a strong endpoint inventory agent, but it should be designed as a managed, offline-capable service rather than a collection script.

Endpoint	Transport	Control plane	Data platform
Collectors + scheduler + queue	mTLS HTTPS + retry	FastAPI + policies + enrolment	Normalization + correlation + PostgreSQL/search

3.1 Recommended endpoint modules

Module	Responsibility
Identity	Generate and securely retain a device ID and enrolment credentials.
Configuration	Validate signed policies, intervals, enabled collectors and limits.
Scheduler	Run collectors with jitter, timeout, concurrency and resource controls.
Collectors	Implement common and OS-specific inventory or event adapters.
Local store	Queue unsent payloads, snapshots and retry state in SQLite.
Transport	Compress, authenticate and upload batches; process acknowledgements.
Updater	Perform signed updates, health checks, rollback and version reporting.

3.2 Plugin-style collector contract

```
from abc import ABC, abstractmethod

class Collector(ABC):
    name: str
    schema: dict[str, str]

    @abstractmethod
    def collect(self, constraints=None) -> list[dict]:
        ...
```

A registry maps policy names to collector implementations. This provides an osquery-like extension model without requiring a SQL layer in the first release.

```
COLLECTORS = {
    'system_info': SystemInfoCollector(),
    'processes': ProcessCollector(),
    'interfaces': InterfaceCollector(),
    'connections': ConnectionCollector(),
}
```

4. Python Collector Examples

4.1 Cross-platform system inventory with psutil

```
import platform, socket, psutil

def collect_system_info():
    return {
        'hostname': socket.gethostname(),
        'os': platform.system(),
        'os_release': platform.release(),
        'architecture': platform.machine(),
        'cpu_logical_count': psutil.cpu_count(logical=True),
        'cpu_physical_count': psutil.cpu_count(logical=False),
        'memory_bytes': psutil.virtual_memory().total,
        'boot_time': psutil.boot_time(),
    }
```

4.2 Running processes

```
import psutil

def collect_processes():
    rows = []
    for proc in psutil.process_iter(
        ['pid', 'ppid', 'name', 'exe', 'username', 'create_time']
    ):
        try:
            rows.append(proc.info)
        except (psutil.NoSuchProcess, psutil.AccessDenied,
                psutil.ZombieProcess):
            continue
    return rows
```

4.3 Network connections

```
def collect_connections():
    rows = []
    for conn in psutil.net_connections(kind='inet'):
        rows.append({
            'family': str(conn.family),
            'type': str(conn.type),
            'local_address': str(conn.laddr) if conn.laddr else None,
            'remote_address': str(conn.raddr) if conn.raddr else None,
            'status': conn.status,
            'pid': conn.pid,
        })
    return rows
```

Privilege note

Some processes, connections, user stores and security events are unavailable to an unprivileged agent. Run only with the minimum permissions required by the enabled collectors.

4.4 Windows installed-software collector

```
import winreg

UNINSTALL_PATHS = [
    r'SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall',
    r'SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall',
]

def read_value(key, name):
    try: return winreg.QueryValueEx(key, name)[0]
    except OSError: return None

def collect_windows_software():
    rows = []
    for hive in (winreg.HKEY_LOCAL_MACHINE, winreg.HKEY_CURRENT_USER):
        for path in UNINSTALL_PATHS:
            try: parent = winreg.OpenKey(hive, path)
            except OSError: continue
            for i in range(winreg.QueryInfoKey(parent)[0]):
                try:
                    key = winreg.OpenKey(parent, winreg.EnumKey(parent, i))
                    name = read_value(key, 'DisplayName')
                    if name:
                        rows.append({'name': name,
                                    'version': read_value(key, 'DisplayVersion'),
                                    'publisher': read_value(key, 'Publisher'),
                                    'source': 'windows_registry'})
                except OSError: continue
    return rows
```

This is only one source. A complete Windows implementation must reconcile machine-wide and per-user registry entries, MSI products, Microsoft Store packages and portable applications.

5. Local Queue, Offline Operation and Change Detection

5.1 Durable local queue

```
CREATE TABLE outbound_events (
  id TEXT PRIMARY KEY,
  device_id TEXT NOT NULL,
  collector TEXT NOT NULL,
  collected_at TEXT NOT NULL,
  payload TEXT NOT NULL,
  attempts INTEGER DEFAULT 0,
  next_attempt_at TEXT,
  status TEXT DEFAULT 'pending'
);
```

- Write the collection result before attempting transmission.
- Compress and upload bounded batches through authenticated HTTPS.
- Delete or archive data only after an explicit server acknowledgement.
- Retry with exponential backoff and jitter; enforce retention and disk limits.
- Protect the queue against malformed payloads and partial database corruption.

5.2 Domain-aware inventory diffs

A full-row hash is useful for initial comparison, but production diffs should use natural keys. For software, a key might combine product name, publisher, architecture and installation scope. A version change can then be represented as an update rather than a removal plus addition.

```
def diff_by_key(previous, current, key_fn):
    old = {key_fn(x): x for x in previous}
    new = {key_fn(x): x for x in current}
    return {
        'added': [new[k] for k in new.keys() - old.keys()],
        'removed': [old[k] for k in old.keys() - new.keys()],
        'updated': [
            {'before': old[k], 'after': new[k]}
            for k in old.keys() & new.keys() if old[k] != new[k]
        ],
    }
```

6. Real-Time Telemetry: Where Python Becomes Difficult

Requirement	Python feasibility	Primary concern
Hardware and OS inventory	High	Normalizing vendor and platform variations
Installed software	High	Multiple package sources and per-user scope
Users and services	High	Permissions and directory integrations
Periodic processes/ports	High	Short-lived activity can be missed
Filesystem changes	High	Watcher overflow, recursion and exclusions
Process start/exit events	Medium	Needs ETW, audit/eBPF or native callbacks
Socket lifecycle events	Medium–Low	Kernel/native event source required
Tamper-resistant security agent	Low	Signing, self-protection and privileged code

Libraries such as psutil and watchdog are valuable building blocks, but they are not substitutes for ETW on Windows, audit/eBPF on Linux, or EndpointSecurity/FSEvents on macOS when complete real-time visibility is required.

7. Production Security and Operations

Control area	Minimum expectation
Device identity	Random enrolment UUID plus device certificate; do not use MAC address as the permanent identity.
Transport	TLS with server validation; preferably mutual TLS and certificate rotation.
Policy	Signed configuration with schema validation, expiry, versioning and rollback.
Execution	Allow-listed collectors, timeouts, result caps, concurrency and resource budgets.
Packaging	Windows code signing, Linux packages, macOS signing and notarization.
Updates	Signed artifacts, staged rollout, health verification and automatic rollback.
Privacy	Data minimization, field redaction, tenant isolation, retention and audit history.
Resilience	Offline queue, bounded storage, retry backoff and corruption recovery.

8. Build-versus-Integrate Decision

Option	Advantages	Disadvantages	Fit
Rebuild fully in Python	Complete product control	Large cross-platform effort; security and event gaps	Only for narrow inventory scope
Use osquery directly	Mature tables and broad OS coverage	Requires fleet integration and normalization	Strong for endpoint extraction
Hybrid: osquery + Python	Fastest path with differentiated control plane	Two components to package and operate	Recommended for Inventory Online

Decision

Adopt the hybrid model. Use osquery as the endpoint instrumentation layer and Python/FastAPI for enrolment, query policy, ingestion, normalization, correlation, compliance and portal services.

9. Recommended Inventory Online Architecture

Endpoint layer	Edge/control layer	Platform layer
osqueryd + signed configuration + local buffering	Python/FastAPI collector gateway + enrolment + policy	Normalization → asset correlation → unified inventory DB → search/reporting APIs

9.1 Suggested service boundaries

- Endpoint Manager: enrolment, credentials, heartbeat, version and health.
- Query Policy Service: schedules, packs, targeting, signing and staged rollout.
- Inventory Collector API: authenticated ingestion, validation, acknowledgement and rate controls.
- Normalization Engine: map OS-specific fields into the common inventory model.
- Asset Correlation Engine: merge observations into stable devices and resolve identity changes.
- Unified Inventory Store: current state, historical changes and provenance.
- Search and Reporting: user-facing APIs, filters, exports, compliance and alerts.

9.2 Delivery roadmap

Phase	Scope	Exit criterion
1. Proof of concept	Windows/Linux inventory: system, software, users, services, ports	Results compared with osquery and normalized
2. Managed agent	Enrolment, signed policy, queue, retry, heartbeat, packaging	Reliable operation through disconnects and restarts
3. Platform integration	Collector API, normalization, correlation and inventory DB	Stable device records and change history
4. macOS + scale	macOS collectors, fleet targeting, performance and staged rollout	Cross-platform pilot at representative scale
5. Event telemetry	Selected file/process/socket events based on product need	Measured event completeness and bounded overhead

10. Practical Conclusion

The easiest part of osquery to reproduce in Python is periodic device inventory. The difficult engineering is the mature cross-platform abstraction, real-time event capture, privileged access, low resource usage, secure fleet operation and years of platform-specific edge cases.

For Inventory Online, Python should own the product intelligence and orchestration. osquery should remain the instrumentation engine unless the intended collector scope is deliberately narrow and limited to periodic inventory.

Sources and Further Reading

- **osquery documentation:** <https://osquery.readthedocs.io/>
- **osquery GitHub repository:** <https://github.com/osquery/osquery>
- **osquery pub-sub event framework:** <https://github.com/osquery/osquery/blob/master/docs/wiki/development/pubsub-framework.md>
- **osquery Python/Thrift extension interface:** <https://github.com/osquery/osquery-python/blob/master/osquery.thrift>
- **osquery table schema:** <https://osquery.io/schema/>